1 Context-Free Grammars

A formal grammar is considered "context free" when its production rules can be applied regardless of the context of a nonterminal. It does not matter which symbols the nonterminal is surrounded by; the single nonterminal on the left-hand side can always be replaced by the right-hand side.

$\Sigma = \{0, 1\}, V = \{S\}$.

Production rules:

$$S \to 0S1$$
$$S \to \epsilon$$

Example Derivation (a string over Σ):

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$$

Definition A CFG is a 4-tuple (V, Σ, P, S), where:

- V is a finite set of *variables*,
- Σ is a finite set of *terminals*,
- P is a set of *production rules*, and
- $S \in V$ is the *start symbol*.

Remark In a CFG, the left side of production rules is always a **single** variable.

Example $V = \{S\},\ \Sigma = \{0, 1\},\ P = \{S \to 0S1,\ S \to \epsilon\}$. Alternatively, $P = \{S \to 0S1 \mid \epsilon\}$.

Definition The language generated by a grammar G is $L(G) = \{w \mid w \in \Sigma^*, S \overset{*}{\Longrightarrow} w\}$

Example For the previous grammar $\{S \to 0S1 \mid \epsilon\}$, we have $L(G) = \{0^n 1^n \mid n \geq 0\}$.

Example $L = \{w \in \{a, b\}^* \mid n_a(w) = n_b(w)\}$

- $\Sigma = \{a, b\}$
- $V = \{S, A, B\}$
- P :

$$S \to aB \mid bA \mid \epsilon$$
$$A \to aS \mid bAA$$
$$B \to bS \mid aBB$$

Example Assume G with $\Sigma = \{0, 1\},\ V = \{S\}$ and $P:\ S \to 0S1 \mid \epsilon$

Claim 1.1 $L(G) = \{0^n 1^n \mid n \geq 0\}$

Proof

1. If $S \overset{*}{\Longrightarrow} w$, for $w \in \{0, 1\}^*$ then $w = 0^n 1^n$ for some $n \geq 0$.
2. If $w = 0^n 1^n$ for some $n \geq 0$, then $S \overset{*}{\Longrightarrow} w$.

First part: Using proof by induction on the number of steps in the production:
Basis: $S \Rightarrow \epsilon = 0^0 1^0$ ✓.
Inductive hyptothesis: If $S \stackrel{*}{\Longrightarrow} w'$, then $w' = 0^n 1^n$ for all derivations from S with k or fewer steps.
Induction step: Suppose $S \stackrel{*}{\Longrightarrow} w$ in $k+1$ steps. The derivation must start with ($k+1 > 1$):

$$S \to 0S1 \stackrel{*}{\Longrightarrow} 0w'1$$

So, for some w', $S \stackrel{*}{\Longrightarrow} w'$ in k steps. Then, by the inductive hypothesis $w' = 0^n 1^n$ for some n. Hence $w = 0^{n+1}1^{n+1} \in L(G)$.

Second part: Using proof by induction on the length of w:
Basis: $w = \epsilon$, then $S \Rightarrow \epsilon$.
Inductive hypothesis : If $S \stackrel{*}{\Longrightarrow} w$ with $|w| \leq k \geq 1$, then w can be derived from S.
Induction step: Let $w = 0^n 1^n, n \geq 1$ with $|w| = k+1 \geq 2$. Consider $w = 0w'1$, where $w' = 0^{n-1}1^{n-1}$ and w' can be derived from S by the inductive hypothesis. Then, as w can be derived from S by first applying:

$$S \to 0S1 \stackrel{*}{\Longrightarrow} 0w'1$$

Example Assume G with $\Sigma = \{a, b\}$, $V = \{S, A, B\}$ and P :

$$\begin{array}{l} S \to aB \mid bA \mid \epsilon \\ A \to aS \mid bAA \\ B \to bS \mid aBB \end{array}$$

Claim 1.2 $L(G) = \{w \in \{a, b\}^* \mid n_a(w) = n_b(w)\}$

Proof

1. If $S \stackrel{*}{\Longrightarrow} w$, then $n_a(w) = n_b(w)$.
2. If $n_a(w) = n_b(w)$, then $S \stackrel{*}{\Longrightarrow} w$.

First part: Using proof by induction on the number of steps $k \geq 1$ in the production.
Inductive hypothesis: $k \leq n$

- If $S \stackrel{k}{\Longrightarrow} w$, then $n_a(w) = n_b(w)$.
- If $A \stackrel{k}{\Longrightarrow} w$, then $n_a(w) = n_b(w) + 1$.
- If $B \stackrel{k}{\Longrightarrow} w$, then $n_a(w) + 1 = n_b(w)$.

Basis:

- $k = 1$: $S \to \epsilon$; $n_a(w) = n_b(w)$ ✓
- $k = 2$: $A \to aS \to a$; $n_a(w) = n_b(w) + 1$ ✓
- $k = 2$: $B \to bS \to b$; $n_a(w) + 1 = n_b(w)$ ✓

Induction step:
Suppose $S \stackrel{*}{\Longrightarrow} w$ in $n+1$ steps. Then, the derivation begins with either of the following:

- $S \to aB \stackrel{*}{\Longrightarrow} aw'$: $B \stackrel{*}{\Longrightarrow} w'$ in n steps; hence, $n_a(w') + 1 = n_b(w')$ by the I.H.
- $S \to bA \stackrel{*}{\Longrightarrow} bw'$: $A \stackrel{*}{\Longrightarrow} w'$ in n steps; hence, $n_a(w') = n_b(w') + 1$ by the I.H.

Either way, $n_a(w) = n_b(w)$.

Left as exercise:
Suppose $A \stackrel{*}{\Longrightarrow} w$ in $n+1$ steps. Show $n_a(w) = n_b(w) + 1$.
Suppose $B \stackrel{*}{\Longrightarrow} w$ in $n+1$ steps. Show $n_a(w) + 1 = n_b(w)$.

Second part: Using proof by induction on the length of w with $|w| = k \geq 0$.
Inductive hypothesis: $|w| = k \leq n$

- If $n_a(w) = n_b(w)$, then $S \stackrel{*}{\Longrightarrow} w$.
- If $n_a(w) = n_b(w) + 1$, then $A \stackrel{*}{\Longrightarrow} w$.
- If $n_a(w) + 1 = n_b(w)$, then $B \stackrel{*}{\Longrightarrow} w$.

Basis:

- $k = 0$: $w = \epsilon$, $S \to \epsilon$ ✓
- $k = 1$: $w = a$, $A \to aS \to a$ ✓ or $w = b$, $B \to bS \to b$ ✓

Induction step: Let $n_a(w) = n_b(w)$ with $|w| = n + 1$. Then, we have either of the following:

- $w = aw'$: $n_a(w') + 1 = n_b(w)$. By the I.H., $B \stackrel{*}{\Longrightarrow} w'$. Then, we can derive w from S as: $S \to aB \stackrel{*}{\Longrightarrow} aw'$.
- $w = bw'$: $n_a(w') = n_b(w) + 1$. By the I.H., $A \stackrel{*}{\Longrightarrow} w'$. Then, we can derive w from S as: $S \to bA \stackrel{*}{\Longrightarrow} bw'$.

Left as exercise:
Suppose $n_a(w) = n_b(w) + 1$. Show $A \stackrel{*}{\Longrightarrow} w$.
Suppose $n_a(w) + 1 = n_b(w)$. Show $B \stackrel{*}{\Longrightarrow} w$.

More examples

- $\Sigma = \{(,)\}$
  $L = \{w \in \Sigma^* \mid w \text{ is a string of properly nested parentheses}\}$.
  E.g., $(), ()(), (()()), \ldots$

$$G = (\{S\}, \Sigma, P, S)$$
$$P : S \to (S) \mid SS \mid \epsilon$$

- $\Sigma = \{\times, +, a, b, 0, 1\}$
  L: Set of arithmetic expressions with variable names over $a, b, 0, 1$, where a variable name must begin with an alphabetical character.

$$V = \{E, I\}\ (Expressions\ and\ Identifiers)$$
$$G = (V, \Sigma, P, E)$$

$$P : E \to E \times E \mid E + E \mid I$$
$$I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$$

1.1 Derivation Trees (Parse Trees)

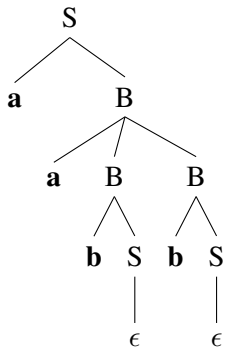
Take:

$$S \to aB \mid bA \mid \epsilon$$
$$A \to aS \mid bAA$$
$$B \to bS \mid aBB$$

Consider $w = aabb$:

$$S \to aB \to aaBB \to aabSB \to aabB \to aabb$$

The derivation tree for this derivation:



Ambiguity: informally, you can derive the same string in multiple different ways.

Example Let $G = \{\{E\}, \{\times, +, a, b\}, P, E\}$ and $P: \ E \to E + E \mid E \times E \mid a \mid b$.
Consider $a + b \times a$, which G can derive ambiguously (has two different parse trees) as follows:

$$E \to E + E \to a + E \to a + E \times E \to a + b \times a$$

$$E \to E \times E \to E + E \times E \to a + b \times a$$

But not every different derivation has an ambiguity (different parse trees). For instance, $a + b$:

$$E \to E + E \to a + E \to a + b$$

$$E \to E + E \to E + b \to a + b$$

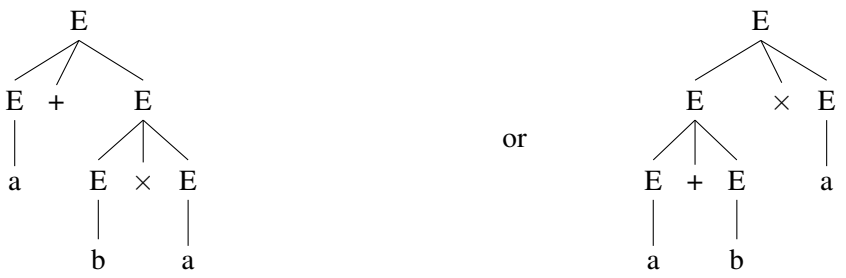
Definition A derivation is "leftmost" if at every step we replace the leftmost variable.

Definition Ambiguity: A string is "ambiguously" in a CFG G if it has two or more different leftmost derivations. Grammar G is "ambiguous" if it generates some string ambiguously.

Example The grammar G above is ambiguous since $a + b \times a$ is derived ambiguously.

Theorem 1.3 *A string is ambiguous iff it has two different parse trees.*

Example $a + b \times a$:



Note that some languages derived by an ambiguous grammar can also be derived by a non-ambiguous grammar as well.

Example $G_2 = \{\{\text{EXPR}, \text{TERM}, \text{FACTOR}\}, \{\times, +, a, b\}, P, \text{EXPR}\}$.

P :

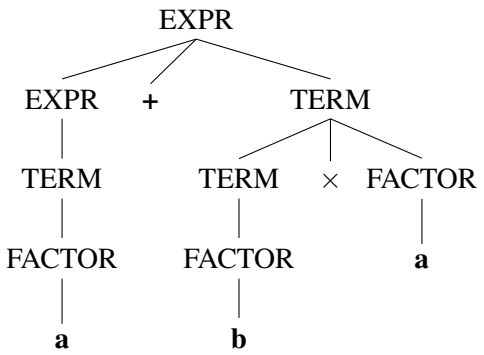
$$\text{EXPR} \to \text{EXPR} + \text{TERM} \mid \text{TERM}$$

$$\text{TERM} \to \text{TERM} \times \text{FACTOR} \mid \text{FACTOR}$$

$$\text{FACTOR} \to a \mid b$$

Consider $a + b \times a$:

$$\text{EXPR} \to \text{EXPR} + \text{TERM} \to \text{TERM} + \text{TERM} \to \text{FACTOR} + \text{TERM} \to a + \text{TERM} \overset{*}{\Longrightarrow} a + b \times a$$



Definition A context-free language (CFL) is called "inherently ambiguous" if it can be produced only by ambiguous grammars.

Example $L = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$. Consider $w = a^n b^n c^n$ for some $n \geq 0$,

1.2 Regular Grammars

If all productions of a CFG G are of the form

$$A \to wB \quad or$$

$$A \to w$$

where $A, B \in V$, $w \in \Sigma^*$, G is called a "right-linear grammar". Similarly, if all productions are of the form

$$A \to Bw \quad or$$

$$A \to w$$

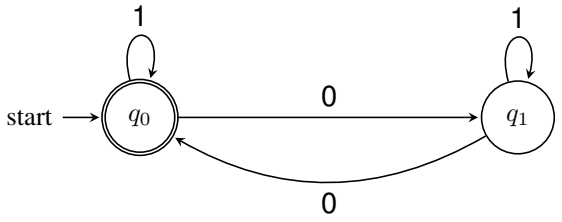
where $A, B \in V$, $w \in \Sigma^*$, G called a "left-linear grammar".

If a grammar is left-linear or right-linear, it is called "regular".

Theorem 1.4 *A language is regular iff it can be produced by a regular grammar.*

Example $L = \{w \in \{0,1\}^* | n_0(w) \; is \; even\}$.

DFA:



RG:

$$S \to 0A \mid 1S \mid \epsilon$$

$$A \to 0S \mid 1A$$

1.3 Simplification of a CFG

We can simplify a grammar by elimination in the following order:

1. ϵ productions - Algorithm 1
2. unit productions - Algorithm 2
3. useless variables - Algorithm 3
4. unreachable symbols - Algorithm 4

1.3.1 Removing ϵ-productions

If $\epsilon \notin L(G)$, we can eliminate all ϵ-productions.

Definition If $A \stackrel{*}{\Longrightarrow} \epsilon$, then A is "nullable".

Algorithm 1 Remove ϵ-productions

if $A \to x_1 x_2 \dots x_n \in P$ **then**
  add $A \to \alpha_1 \alpha_2 \dots \alpha_n$ to P' where not all α_is are ϵ:
  if x_i is not nullable **then**
    $\alpha_i = x_i$
  end if
  if x_i is nullable **then**
    $\alpha_i = x_i$ or $\alpha_i = \epsilon$
  end if
end if

Example P:

$$S \to ABaC$$
$$A \to BC$$
$$B \to b \mid \epsilon$$
$$C \to c \mid \epsilon$$

Nullable: A, B, C

P':

$$S \to ABaC \mid BaC \mid AaC \mid ABa \mid aC \mid Ba \mid Aa \mid a$$
$$A \to BC \mid C \mid B$$
$$B \to b$$
$$C \to c$$

1.3.2 Unit productions

Remove productions of the form $A \to B$

Algorithm 2 Remove unit productions

P' = all non-unit productions of P
if $A \stackrel{*}{\Longrightarrow} B$ for $A, B \in V$ and $B \to \alpha$ is a non-unit production **then**
  add $A \to \alpha$ to P'
end if

Example P :

$$S \to aBCD$$
$$B \to bB$$
$$B \to C$$
$$C \to D$$
$$D \to d$$

Note that $B \stackrel{*}{\Longrightarrow} C$ and C has no non-unit productions. For $C \stackrel{*}{\Longrightarrow} D$ and $B \stackrel{*}{\Longrightarrow} D$, $D \to d$.
Initialization: $P' : S \to aBCD,\ \ B \to bB,\ \ D \to d.$
Second step: $P' : S \to aBCD,\ \ B \to bB \mid d,\ \ C \to d,\ \ D \to d.$

1.3.3 Eliminating useless variables

In a grammar $G = \{V, \Sigma, P, S\}$, a variable X is useful if there is a derivation

$$S \stackrel{*}{\Longrightarrow} \alpha X \beta \stackrel{*}{\Longrightarrow} w$$

for some $\alpha,\ \beta$; and $w \in \Sigma^*$. Otherwise, X is said to be "useless".

Algorithm 3 Remove useless variables

Put every variable A into V' such that $A \to w$ for some $w \in \Sigma^*$
repeat
  if $(B \to x_1 x_2 \dots x_n) \in P$ where each $x_i \in (\Sigma \cup V')$ **then**
    put B into V'
  end if
until no new variables are added to V'
Remove all productions that contain useless variables

Example $V = \{S, A, B, C\}, \Sigma = \{a, b, c\}$
P:

$$S \to AB \mid aA$$
$$A \to a$$
$$C \to ca$$

Initialization:

$$V' = \{A, C\}$$

Loop: S will be added to V' because aA is terminal since we know A will produce terminal string ($A \in V'$).

$$V' = \{A, C, S\}$$

Stop. B is useless. Remove $S \to AB$ from P.

1.3.4 Remove unreachable symbols

Example $V = \{A, C, S\}$,
P:

$$S \to aA$$
$$A \to a$$
$$C \to ca$$

Initialization: $V' = \{S\}, \Sigma' = \emptyset$
Iteration 1: $V' = \{S, A\}, \Sigma' = \{a\}$
Iteration 2: $\times$
$V' = \{S, A\}, \Sigma' = \{a\}, P' : S \to aA,\ \ A \to a.$

Algorithm 4 Remove unreachable symbols

Place S in V'
repeat
  if $A \in V'$ and $A \to \alpha \in P$ **then**
    add all variables in α to V'
    add all terminals in α to Σ'
  end if
until no new variable or terminal is added

Remark To remove useless symbols, first apply Algorithm 3, then Algorithm 4. If you reverse the order, it may not work.

Example $S \to AB \mid c$, $A \to a$
Alg 3: $S \to c$, $A \to a$
Alg 4: $S \to c$
If we do it the other way:
Alg 4: $S \to AB \mid c$, $A \to a$
Alg 3: $S \to c$, $A \to a$, $\times$

1.4 Chomsky normal form (CNF)

If all productions in a CFG G are of the form:

$$A \to BC \quad or$$

$$A \to a$$

where $A, B, C \in V$ and $a \in \Sigma$, then G is in *Chomsky normal form*.

Theorem 1.5 *Any CFL without ϵ can be generated by a grammar in CNF. Hence, every CFG can be converted into CNF.*

1.4.1 Conversion into CNF

- First, simplify the grammar and obtain $G = (V, \Sigma, P, S)$ without any useless variables/symbols, ϵ-, or unit productions.
- If there is a single symbol on the right, it is a terminal; i.e., no unit productions on the left.
- Consider a production $A \to X_1 \dots X_m$ ($m \geq 2$).
  - If X_i is a terminal a, introduce variable C_a and production $C_a \to a$; replace X_i by C_a.
  - For each production $A \to B_1 \dots B_m$, $m > 2$, add new variable $D_1, D_2, \dots, D_{m-2}$, and replace this production with:

$$A \to B_1 D_1$$

$$D_1 \to B_2 D_2$$

$$D_2 \to B_3 D_3$$

$$\dots$$

$$D_{m-2} \to B_{m-1} B_m$$

Example

$$S \to aB \mid bA$$
$$A \to aS \mid bAA \mid a$$
$$B \to bS \mid aBB \mid b$$

$$S \to C_aB \mid C_bA$$
$$A \to C_aS \mid C_bAA \mid a$$
$$B \to C_bS \mid C_aBB \mid b$$
$$C_a \to a$$
$$C_b \to b$$

$$S \to C_aB \mid C_bA$$
$$A \to C_aS \mid C_bD_1 \mid a$$
$$B \to C_bS \mid C_aD_2 \mid b$$
$$D_1 \to AA$$
$$D_2 \to BB$$
$$C_a \to a$$
$$C_b \to b$$

If $\epsilon \in L(G)$ then we allow $S \to \epsilon$ for the start variable S.
Construction: After converting G into CNF, add a new start variable S_0 and $S_0 \to S \mid \epsilon$.

Remark Parsing a grammar in CNF can be achieved using the *CYK Algorithm* (Cocke-Younger-Kasami), which employs *bottom-up parsing* and dynamic programming.

1.5 Greibach normal form (GNF)

If all productions in a CFG G are of the form:

$$A \to a\alpha$$

where $\alpha \in V^*$ and $a \in \Sigma$, then G is in *Greibach normal form*.

Theorem 1.6 *Every CFL can be generated by a grammar in GNF.*

Example

$$S \to aB \mid bA$$
$$A \to aS \mid bAA \mid a$$
$$B \to bS \mid aBB \mid b$$

Remark Given a grammar in GNF and a derivable string in the grammar with length n, any *top-down parser* will halt at depth n.

1.6 Parsing

Definition Top-down parsing: Look at the highest level of the parse tree and work down using the rewriting rules of the grammar. Easy to construct manually.

- Recursive descent parsers
- Predictive parsers
- Nonrecursive predictive parsers. LL parser: parses from left-to-right; i.e., leftmost derivation.

Definition Bottom-up parsing: Start from the leaves of the parse tree and work up until you reach the root. They can handle a larger class of grammars.

- LR parser: rightmost derivation in reverse. Types of LR parsers are LALR and SLR parsers. Guaranteed linear run time.

Definition LL(1) grammar: grammars that can be parsed with an LL parser with 1-step lookahead. Most programming languages have LL(1) grammars.

Example Consider the following grammar for balanced parentheses: $G = \{V, \Sigma, P, B\}$, $V = \{B, R\}$, P:

$$B \to (RB \mid \epsilon$$

$$R \to) \mid (RR$$

$w = (()())()$:

$$B \to (RB \to ((RRB \to (()RB \to (())B \to (())(RB \to (())()B \to (()())()$$

1.7 Chomsky hierarchy

Classes of normal grammars:

1. Type-0 grammars, or *unrestricted grammars* include all formal grammars that generate all languages that can be recognized by a Turing machine (*recursively enumerable languages*).
2. Type-1 grammars, or *context-sensitive grammars* generate context-sensitive languages.
3. Type-2 grammars, or *context-free grammars* generate context-free languages.
4. Type-3 grammars, or *regular grammars* generate regular languages.

Hierarchy: Type-3 $\subset$ Type-2 $\subset$ Type-1 $\subset$ Type-0.

Grammar	**Languages**	**Automaton**	**Production rules**
Type-0	Recursively enumerable	Turing machine	$\alpha \to \beta$ (no restrictions)
Type-1	Context-sensitive	Linear-bounded non-deterministic Turing machine	$\alpha A \beta \to \alpha \gamma \beta$
Type-2	Context-free	Non-deterministic pushdown automaton	$A \to \gamma$
Type-3	Regular	Finite state automaton	$A \to a$ and $A \to aB$